ConfigStream

Scalable Configuration Change Storage and Streaming Analytics

Phạm Công Hoàng Lê Tiến Hợp
Mai Vũ Hoàng Nguyễn Đức Dũng
Group 4 · Big Data Storage & Processing

First milestone checkpoint · 26 October 2026

The problem: configuration changes over time

Router configuration: version 10

```
hostname router-001
logging enabled
telnet disabled
```

Version 11

```
hostname router-001
logging disabled
telnet enabled
```

What changed, when, and on which asset?

A current configuration file alone cannot answer all three.

Illustrative input from the executable demo: processing/demo.py.

Four datasets, four meanings of change

Terraform / TerraDS: versions

Infrastructure code stored in Git.

v1 → **v2**: which lines changed?

RPKI: observed daily states

Observed route-authorization records.

Day 1 → **day 2**: which members differ?

RIS: routing events

BGP announcements and withdrawals.

Announce → **withdraw**: what happened?

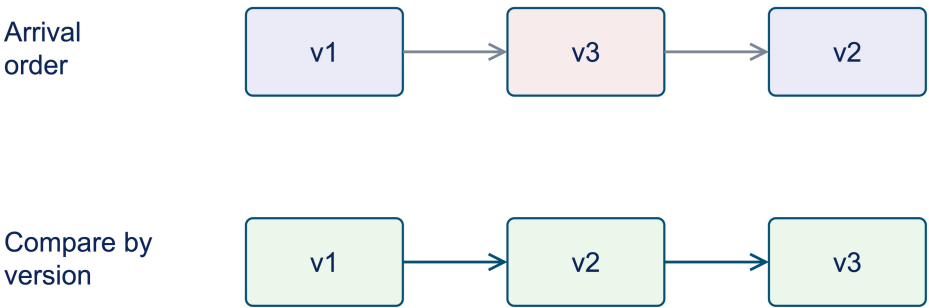
DNS: measurements

Recorded DNS query and response data.

Name → **IP**: what was observed, when?

Conceptual examples. Sources and accepted scopes: frontend/public/research/v1/catalog.json.

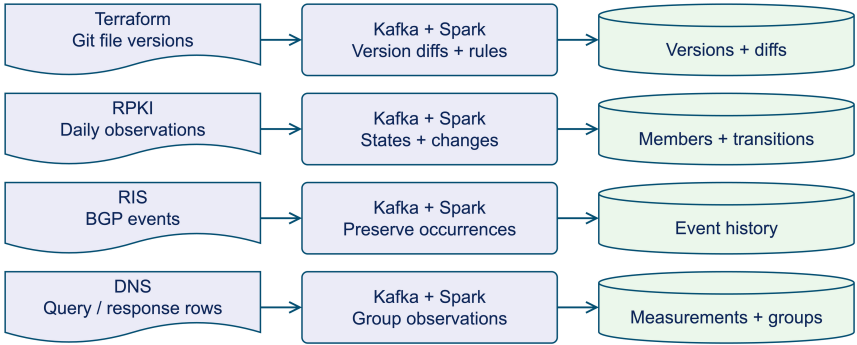
Configuration example: handling late arrivals



Compare **v1** → **v2** and **v2** → **v3**, once both endpoints are available.

processing/state.py; spark/streaming/stateful.py. Online repair is bounded by retained state.

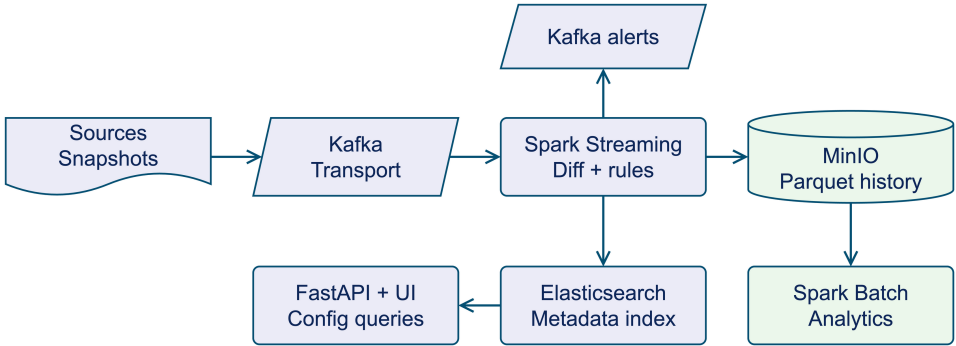
System architecture: four separate processing paths



Kafka + Spark + MinIO (Parquet), with separate schemas and processing.
Config search: Elasticsearch + FastAPI. Dashboard: accepted results from all four.

spark/streaming/; spark/rpki_stream.py; spark/ris.py; spark/dns.py; domain architecture docs.

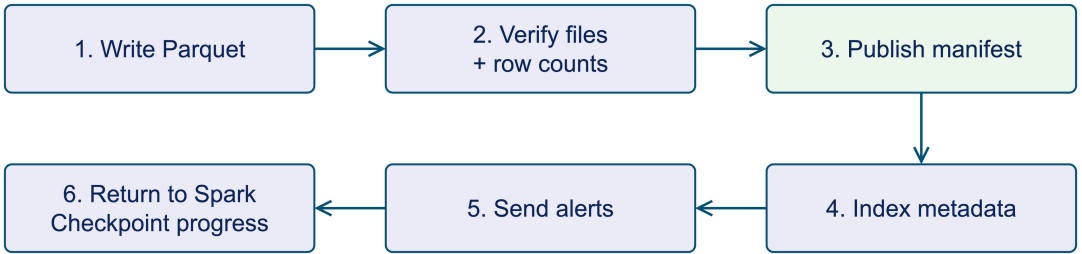
Configuration path: transport, compute, store, serve



Lake writes complete before search indexing and alert publication.
Search can be rebuilt from committed Parquet; batch analytics reads the lake.

Lake-enabled configuration path: spark/streaming/outputs.py; search/reindex.py; analytics/main.py.

A commit manifest marks readable history



Readers use committed files. Partial writes stay outside accepted history.

An exact retry verifies and reuses the committed batch.

Current lake-enabled code: spark/streaming/sinks/lake.py; spark/streaming/outputs.py.

No atomic transaction across sinks. Latest integrity changes still need a completed Kubernetes recovery rerun.

Prepared data: accepted scope for each dataset

Dataset	Accepted amount	Selected scope
Terraform	**10,188** Git versions	6,505 file histories / 375 repositories
RPKI	**14** complete daily observations	ripencc.tal, 1–14 Sep 2026
RIS	**4,725,156** event occurrences	rrc00, 1 Sep 2026, 00:00–01:00 UTC
DNS	**2,234,863** raw measurements	Two .li source objects, 1–2 Sep 2026

Controlled synthetic inputs also test retries and late arrivals.
Source manifests and result receipts are versioned in Git.

Accepted evidence: frontend/public/research/v1/catalog.json and its linked validation artifacts. Units differ; do not sum the rows.

Implemented: differences and rule alerts

Diff: version 10 → 11

```
- logging enabled
- telnet disabled
+ logging disabled
+ telnet enabled
```

Rules detect

Logging disabled
High severity

Telnet enabled
Critical severity

Repeated delivery must not invent another logical change.

processing/demo.py; processing/diff.py; rules/default.yaml; tests/unit/test_rules.py.

Progress: runnable code and a recorded pipeline replay

Repository

`generator/`	input data
`processing/`	diff and rules
`spark/`	streaming jobs
`api/`	read API
`frontend/`	dashboard

Small demo, no services required

```
make setup
make processing-demo
```

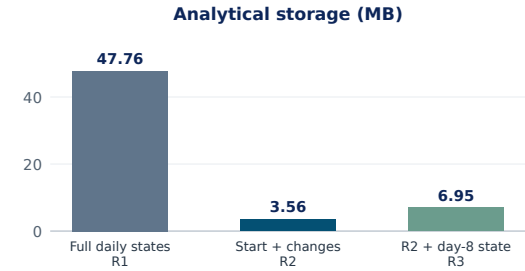
178 core tests passed
State logic, diffs and rules
Rechecked on 25 October 2026

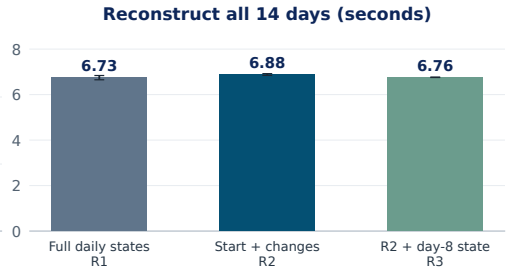
Recorded TerraDS replay: 10,188 snapshots → 3,683 diffs; 29.4 MB Parquet.

Recorded replay: docs/phase12-acceptance.md; github.com/kongwoang/ConfigStream; Makefile; tests/unit/test_state_logic.py, test_diff.py, test_rules.py.

RPKI: less storage, similar reconstruction time

Same 14 daily observations. All three layouts reconstruct the exact same states.





R2 saves 92.6% of analytical storage.

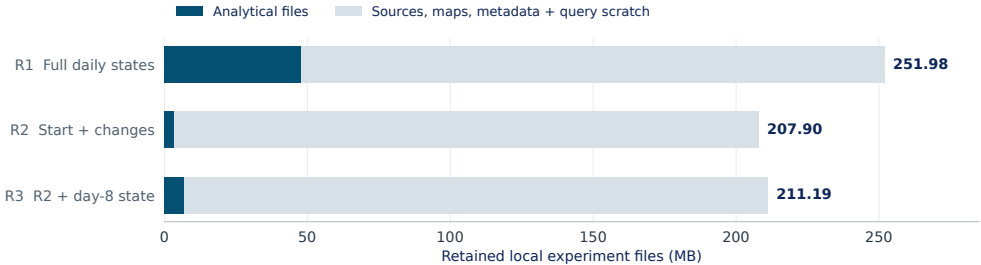
Reconstructing all 14 days still takes about 6.9 seconds.

Recorded local E2R experiment: workloads/results/phase20/rpki-representations.json. MB = 1,000,000 bytes.

Time: median of 3 build medians, each with 3 warm cycles; whiskers show the range of build medians. Independent verification timed separately.

The storage saving depends on what we count

Same RPKI experiment, now including the other retained files.



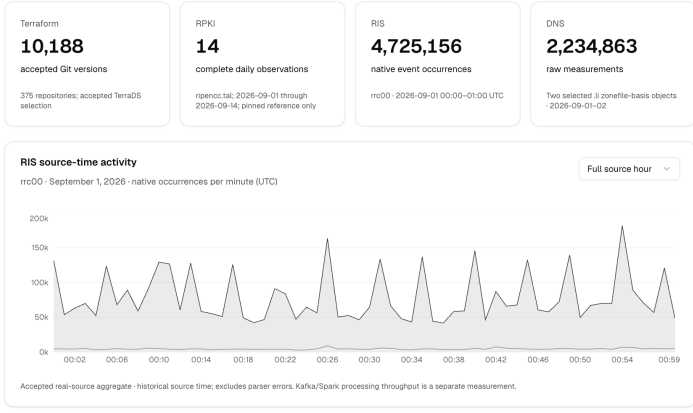
R2 saves 17.5% across measured local experiment files.

Sources, provenance maps and query scratch still occupy most of the space.

workloads/results/phase20/rpki-all-in-storage.json. Median accounted content bytes across 3 builds per layout.

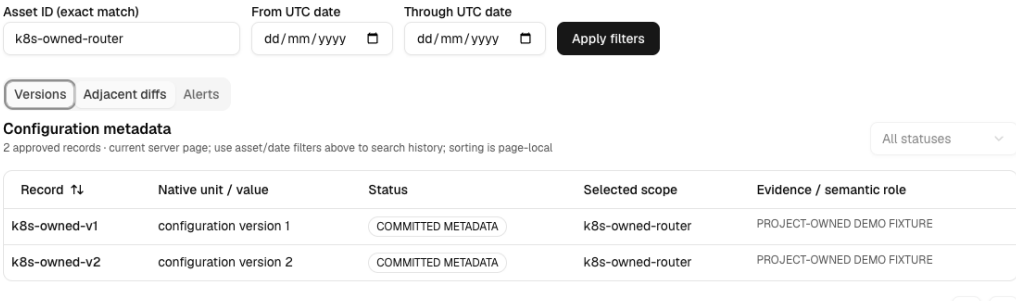
Includes analytical files, sources, coordinate maps, metadata and retained scratch. Excludes deployed Kafka / checkpoint costs and filesystem overhead.

Dashboard: four domains in one evidence overview



Actual frontend captured on 25 October 2026. This overview displays the accepted evidence bundle, not a live replay.

Dashboard: inspect configuration history by asset



1. **Filter** by asset and UTC dates.
2. **Select** versions, adjacent diffs or alerts.
3. **Inspect** safe metadata from the configuration API.

Recorded Phase 20V UI capture, 14 October 2026; docs/phase20v-demo-script.md.

The demonstrated asset is an owned fixture with 2 versions / 1 diff. Raw configuration bodies are not returned.

Checkpoint readiness and next milestone

Requirement	What the group can show
Executable code	Runnable demo, domain pipelines and tests
Prepared data	Terraform, RPKI, RIS, DNS and source manifests
Architecture	Kafka, stateful Spark, committed lake and serving

Next: rehearse the latest integrated demo, repeat recovery checks, and extend workload measurements.

Existing deployment evidence is local / single-node. Scaling and HA remain open.

Thank you — questions?

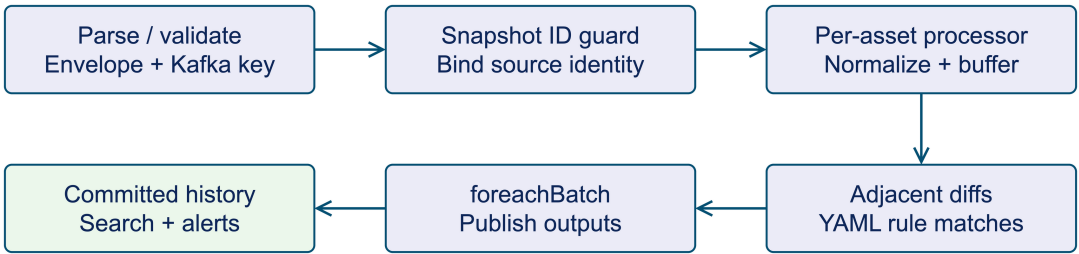
Kafka: an offset is not a configuration version

Three configuration input topics	What the processor reads
`config.assets`: asset metadata	Spark reads `config.snapshots`.
`config.events`: audit events	Kafka keys use `asset_id`.
`config.snapshots`: versioned input	Offsets locate records within a partition.

Partition order preserves delivery order, not source-version order.
No total order or atomic publication across these topics.

messaging/kafka_producer.py; spark/streaming/kafka_source.py; docs/kafka-topics.md.

Spark: identity checks precede per-asset version state



Raw fetched Kafka records have a separate archival branch.
Invalid input and conflicts produce diagnostics. Watermarks and state retention bound repair.

spark/streaming/validation.py, identity.py, stateful.py; processing/state.py.

Current code structure; this diagram does not certify the latest image's Kubernetes recovery.

Parquet layout separates raw evidence and processed history

```
config-lake/
  raw/snapshots/
  processed/snapshots/
  processed/diffs/
  processed/alerts/
  processed/diagnostics/
  _commits/
  analytics/
```

Raw: binary Kafka key/value, topic, partition, offset and timestamp.

Processed: typed snapshots, diffs, alerts and diagnostics.

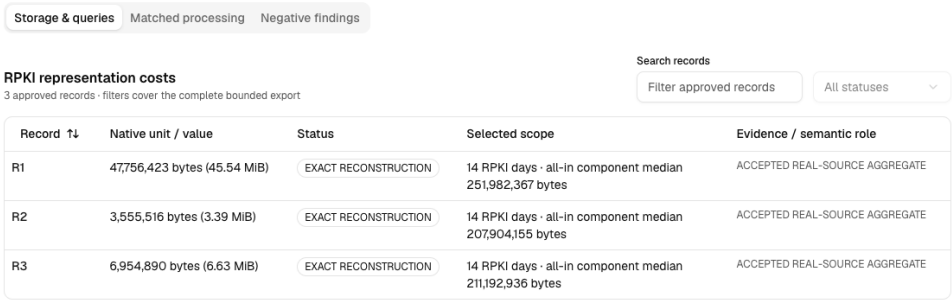
Each configuration batch has a query/batch namespace and UTC date partitions.

Example:

`processed/snapshots/query_id=.../batch_id=.../year=2026/month=10/day=5/`

spark/streaming/sinks/lake.py; docs/data-lake.md. Configuration layout only; other domains use separate temporal paths.

Dashboard: inspect the evidence behind a comparison

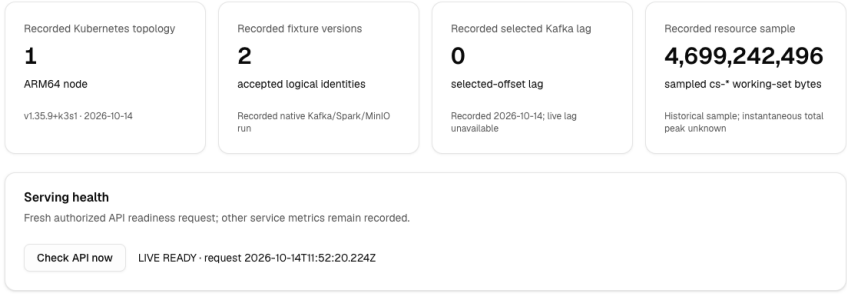


The experiment view exposes storage, query measurements and evidence references.

Compare each number within its declared workload and measurement boundary.

Recorded Phase 20V Experiments capture; accepted RPKI results. Research views read the pinned evidence bundle.

Dashboard: distinguish recorded runs from an API probe



Top cards: recorded Kubernetes, lag and resource evidence.

Check API now: a fresh readiness request when the backend is available.

The READY label in this historical capture applies only to its displayed timestamp.

Recorded Phase 20V Operations capture, 14 October 2026; docs/phase20v-data-contracts.md.

The UI does not expose live Kafka lag or live cluster-wide resource metrics.